

跨境政策研究 Agent：3wagent 课程项目报告

人类定义架构与能力边界、AI Agent 负责开发执行的垂直领域研究工作室

课程名称 大语言模型与信息决策

姓名 睦铭涛、陈泽宇

指导教师 袁坤

学号 2400011018、2400011261

提交日期 2026-06-27

摘要 3wagent 是一个面向中国内地、美国、香港、新加坡四法域的跨境政策研究工作室。它的核心主张是：把一次政策研究中**最值钱的领域判断边界**——何时资金合规优先、何时税务独立分析、哪些来源能支撑结论、输出必须包含哪些要素——由人类显式定义并沉淀为结构化配置与规则；而把检索、时效校验、专项分析、报告生成等**执行性工作**交给以 Claude Code 为载体的 AI Agent 完成。项目刻意不自建 FastAPI / LangChain 分析运行时，而是采用 agent-native 架构：一个 Lead Policy Agent 负责编排，七个专项子代理在既定边界内分工协作。本文从设计目标、能力边界、系统架构、关键工程实现，到「人定义边界、agent 执行」的协作开发过程逐层说明，并以真实跨境案例展示输出契约的落地形态。截至报告时，项目历经 9 个演进阶段、16 次提交，工具与服务层约 2300 行代码，配套 81 个通过的自动化测试。[项目地址](#)

关键词 跨境政策研究；AI Agent；能力边界；人机协作；检索增强（RAG）；可审计

1 引言

1.1 背景与痛点

跨境交易的合规判断天然分散在多个法域、多个监管口径之间。同一笔「向境外支付服务费」，既可能触发外汇管理手续，也可能涉及增值税代扣、企业所得税预提、常设机构判定，还可能因关联关系牵出转让定价。研究者面临三重困难：（1）**来源可信度参差**——官方法规、律所简报、公众号文章混杂，难以快速分级；（2）**法规时效易错**——现行版本、被替代版本、历史版本交织，引用过期条款风险高；（3）**领域边界易混**——外汇、税务、民商法的分析主线常被错误地搅在一起。

1.2 设计理念：人定义边界，agent 负责实现

面对上述痛点，本项目没有选择「再造一个法律科技应用」，而是把工作显式拆成两类（见表 1）：人类作为架构师，负责**定义判断边界与质量契约**；AI Agent 作为执行者，负责**在边界内完成具体研究与开发**。这条主线贯穿全文，也是项目区别于「让大模型自由发挥」的根本所在——Agent 的高效不应以牺牲可审计性为代价，边界由人写下，执行交给机器。

表 1：两类工作的划分

角色	负责什么	沉淀在哪里
人类（架构师）	问题边界、领域判断规则、来源可信分级、输出契约、明确不做什么	config/、principles.md、contracts.md
AI Agent（执行者）	在边界内检索、时效校验、专项分析、报告与工具代码生成	.claude/agents/、.claude/skills/、tools/、mcp_server/

1.3 报告组织

第 2 节给出人类定义的设计目标与能力边界；第 3 节说明 agent-native 系统架构；第 4 节简述由 agent 完成的关键工程实现；第 5 节以版本演进为线索，呈现人机协作的开发过程与人类工作量；第 6 节用真实案例展示领域能力；第 7 节汇报测试、质量与局限；第 8 节讨论经验与反思；第 9 节总结。

2 设计目标与能力边界（人类的领域设计）

项目的智力核心不是某段代码，而是一组**被显式写下来、可审计、可复用的**判断边界。它们集中沉淀在 `config/` 四个 YAML 文件中，作为全局唯一来源；配套的 `principles.md` 进一步写下 7 条操作原则（先识别再行动、子代理只返回证据、结论前必先检索、分析前先校验时效、交付前先校验引用、默认中文输出、够用则不另造框架）。

2.1 问题分类与路由

所有问题先按 `routing.yaml` 归入四类，决定主分析方向：

- **纯粹外汇管理**（SAFE、结售汇、经常项目、资本项目）→ 资金合规
- **纯粹银行合规/AML/制裁**（KYC、反洗钱、OFAC、支付牌照）→ 资金合规
- **纯粹税务**（预提税、利得税、增值税、税收协定、服务费定性）→ 税务
- **民商法规**（公司设立、股权转让、合同、投资准入）→ 民商法

并约定跨领域规则：当税务与资金问题并存、核心是「付款性质判断」时，**税务为主分析、资金合规为辅**。这条看似简单的规则，正是领域专家经验的显式化。

2.2 来源可信分级

所有证据按 `source-levels.yaml` 的 S/A/B/C/D 五级分级（表 2）。关键边界：**C、D 级来源只能作为线索，不得作为最终法律依据**。这把「官方依据」与「专业解读、自媒体传闻」从制度上隔离开。

表 2：来源可信分级（S/A/B 可支撑结论，C/D 仅作参考）

等级	来源类型	示例	可作最终依据
S	法律、法规、条例、官方法律数据库	国家法律法规数据库	是
A	监管机构 / 税局指引、官方 FAQ	SAFE、IRD、IRAS 指引	是
B	官方通函、公告、判例、正式通知	裁判文书网、HKLII	是
C	律所 / 会计师 / 银行专业简报	事务所 alert	否（线索）
D	公众号、媒体文章、个人评论	自媒体解读	否（线索）

2.3 输出契约

`output-contract.yaml` 规定每份结论必须覆盖 5 项核心 + 3 项辅助内容，其中「**涉及现行法规**」为优先级重点：

1. **【问题分类】**——分类、子领域与交易类型；
2. **【结论或考量维度】**——可直接引用的结论，或必须考量的维度；

- 3. 【类案与公开答案】——类似公开判决（含案号、法院、要点、原文）；
- 4. 【涉及现行法规】（重点）——现行有效法规的完整编号清单，含法域、领域、机关、状态；
- 5. 【法规修订关系】——每条现行法规的单层级修订/替代/废止关系。

辅助三项为【风险提示】【结论可靠性】【报告文件】。契约把「研究做到什么程度才算完整」固化下来。

2.4 明确的非目标 (Non-Goals)

人类同样显式定义「现在不做什么」：MVP 阶段不自建分析运行时；来源规模未要求前不引入持久向量库；不做未经引用校验的自动法律结论；不把 C/D 级来源提升为权威。非目标与目标同等重要——它们防止 agent 在执行中越界。

3 系统架构 (agent-native)

3.1 关键架构决策：放弃自建 runtime

项目第一版曾用 Python / FastAPI / LangChain 搭建后端原型，但很快做出最重要的一次转向：移除自建分析运行时，改由 Claude Code / Codex 通用 agent 承载执行。理由有三：通用 coding agent 已具备工具调用、文件读写、检索与任务拆分能力，重写编排框架是重复劳动；个人研究场景中领域规则的稳定性比运行时工程更值钱；规则与子代理以纯文本沉淀，可审计、可 diff、可复用。由此形成原则——规则、技能、子代理够用时，不另造后台框架；现存的 FastAPI dashboard 与 MCP bridge 都只是薄的可选层。

3.2 分层与依赖方向

系统严格遵循「配置与逻辑分离、依赖单向」：config/（策略唯一来源）被 agents/skills 引用，再由 CLAUDE.md 中的 Lead Policy Agent 编排；工具与服务层只读本地产物、不反向污染策略。好处是「改一处生效全局」：新增一类问题分类只需改 routing.yaml，所有引用者自动对齐。

表 3: config/ 四个唯一来源文件

文件	定义内容
routing.yaml	四类问题分类、关键词、funds 子领域、跨领域主辅规则、agent 映射
source-levels.yaml	S/A/B/C/D 五级可信分级，及每级「能否支撑最终结论」标志
jurisdictions.yaml	四法域设置、官方类案数据库入口、各法域来源注册表路径
output-contract.yaml	5 项核心 + 3 项辅助输出节段定义，标注优先级

3.3 角色分工

主会话作为 Lead Policy Agent 负责路由、任务包构造、冲突处理与最终写作；七个子代理按关注点分工，只做自包含专项任务，返回证据而非成稿（图 1）。

3.4 一次请求的流动

用户问题或文档进入后，依次经过：识别法域/领域/付款性质并分类 →（如有文件）解析 → rag-retriever 先查 sources/ 注册表与本地 RAG 索引（含类案库）→

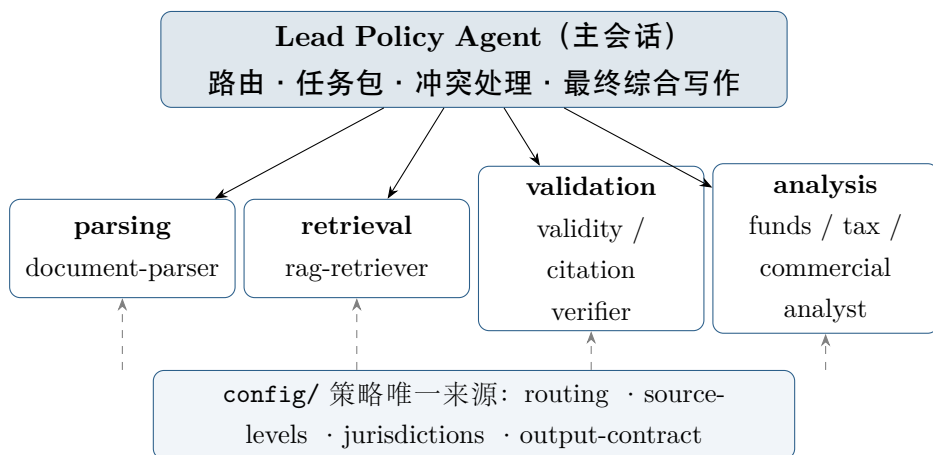


图 1: agent-native 架构: 主会话编排, 子代理按关注点分工, 全部引用 config/ 唯一来源

regulatory-validity-verifier 校验时效与单层级修订关系 → 专项 analyst 在各自边界内分析 → citation-verifier 校验法域、来源等级与结论支撑 → 主会话按输出契约综合成稿并归档。

4 关键工程实现 (agent 完成)

以下模块均由 agent 在人类设定的边界内实现, 强调「可选、轻量、只读」, 不改变 agent-native 主架构。

表 4: 工程层模块与职责

模块	位置	职责
轻量本地 RAG	mcp_server/rag_index.py	SQLite FTS 全文索引; 分块、检索、按来源重组; 保留法域/领域/等级等元数据做 metadata-first 过滤
MCP 检索桥接	mcp_server/server.py	暴露 6 个工具: 注册表检索、文档检索/读取、路由分类、报告列举/读取
来源采集 CLI	tools/ingest_sources.py	从快照、文本文件或注册表元数据构建索引
报告生成	tools/render_report.py 等	Markdown 生成 + PDF 转换, 含 pandoc / weasyprint / reportlab 三级回退
本地 dashboard	server/app.py、viewer/	薄 FastAPI 只读 runs/ 与 reports/; 静态前端轮询展示进度与报告
进度日志	tools/progress.py	写运行状态与分步事件, 供 dashboard 观测
配置/注册表校验	tools/validate_*.py	校验 config/ 与来源注册表的 schema 与一致性

一个体现边界意识的细节: 本地检索坚持「先按法域、领域、来源等级过滤, 再做关键词全文召回」, 且 RAG 只负责证据检索与出处定位, 不直接生成法律结论——把「找证据」与「下判断」分开。

5 人机协作的开发过程

本项目本身就是「人定义边界、agent 执行」的一次实践：人类提出方向与架构决策、定义并不断收紧边界、审阅纠偏；agent 完成重构、补测试、写文档与工具代码。版本日志（CHANGELOG.md）完整记录了 9 个阶段（表 5）。

表 5: 9 个演进阶段：人类决策与 agent 执行的分工

阶段	核心变化	人类决策 / agent 执行
一	项目初始化	人：确定跨境政策研究 Agent 方向
二	Python/LangChain 应用原型	人：验证业务模块边界；agent：搭后端原型
三	框架文档化	人：梳理技术栈与路由优先级
四	agent-native 重构（关键转折）	人：决策放弃自建 runtime；agent：迁移为 rules/skills/subagents
五	报告归档与 PDF 输出	人：定归档契约；agent：实现 reports/ 与转换链
六	输入输出要求细化	人：定 5 项输出、现行法规清单优先；agent：改模板与子代理
七	配置分离与关注点组织	人：定 config 单一来源与依赖方向；agent：抽取 config/、重组目录、补校验与测试
八	轻量本地 RAG 与 MCP 工具	人：定「先过滤再召回、RAG 不下结论」边界；agent：实现 SQLite FTS 与 MCP 工具
九	本地 Dashboard 与服务桥接	人：定「只读观测、不承载分析」边界；agent：实现薄 FastAPI、前端与进度日志

5.1 人类工作量在哪里

表面上「代码由 agent 写」，但真正决定项目形态的是人类的持续判断：四类问题分类、跨领域主辅规则、S/A/B/C/D 分级、5 项输出契约、单层级修订追溯、一系列非目标，以及阶段四「放弃自建 runtime」这一关键架构转向。这些都不是 agent 自发产生的，而是人类把领域经验与工程审美显式写成规则、再约束 agent 执行的结果。可以说，人类的工作量沉淀在 config/ 与 .claude/ 的每一条边界里，而非代码行数里。

5.2 规模量化

为如实反映工作量，给出可核验的客观数据（表 6，均来自仓库实际统计）。

表 6: 项目规模（截至报告时的真实统计）

指标	数量	指标	数量
演进阶段	9	子代理 / 技能	7 / 7
Git 提交	16	配置文件（唯一来源）	4
通过的自动化测试	81	来源注册表（四法域）	4
工具/服务层代码	≈2300 行	MCP 工具	6
测试代码	≈760 行	覆盖法域	4 (CN/US/HK/SG)

6 运行示例（领域能力）

以仓库中 `examples/cn-sg-service-fee-cross-domain` 这一**真实跨领域样例**说明输出形态：中国境内 A 公司向新加坡 B 公司支付技术咨询费 50 万美元，技术人员定期来华、双方存在关联关系，同时涉及外汇手续、税种判断与转让定价。系统按输出契约给出结构化结论（摘录）：

- **问题分类**：跨领域；子领域含 funds-外汇管理、tax-企业所得税、tax-增值税、tax-转让定价；交易为服务贸易。
- **可直接引用的结论**：单笔超等值 5 万美元需先办税务备案；服务部分在境内完成需代扣 6% 增值税；按累计停留天数判断是否构成常设机构（183 天阈值），据此适用 10% 预提或 25% 核定。
- **现行法规清单（重点）**：列出《外汇管理条例》、汇发〔2013〕30 号、企业所得税法及实施条例、财税〔2016〕36 号、中新税收协定、特别纳税调查 2017 年第 6 号公告等 9 项，逐条标注法域、领域、机关与现行状态。
- **修订关系**：例如财税〔2016〕36 号替代财税〔2013〕106 号，标注关系类型与生效日期。
- **风险提示 / 可靠性**：标注服务定性、常设机构、转让定价等风险；说明结论由 S/A 级来源支撑、哪些仍需人工复核。

该样例展示了系统价值：不是给一个含糊答复，而是给一份**可追溯到官方来源、标注了时效与可靠性的结构化研究结论**。说明：样例用于演示分析结构与来源分级方法，实际使用仍需经检索、时效与引用三道校验后方可作为正式依据。

6.1 边界的体现：单一领域 vs 跨领域

与之对照的是 `examples/cn-service-fee-tax` 这一**纯粹税务样例**：服务完全在境外完成、无人员来华，系统据 `routing.yaml` 判为「纯粹税务」，结论聚焦「无需代扣增值税与企业所得税、可税前扣除」，**不强行牵扯外汇管理**。两个样例的对比恰好印证了人类定义的分类边界在执行中真实起作用——单一领域问题独立分析、跨领域问题才按「税务为主、资金为辅」叠加。边界写得清楚，agent 才不会把问题越分越散。

7 测试、质量与局限

7.1 质量保障

项目以轻量但完整的工程纪律保证可维护性：81 个自动化测试覆盖配置、注册表、本地 RAG 索引、MCP 工具、报告渲染、进度日志与 dashboard API，全部通过；`ruff` 静态检查无告警；`validate_config.py` 与 `validate_registry.py` 校验配置与来源注册表的一致性；报告 PDF 转换设计了三级回退，缺少某一工具时自动降级并说明原因。

7.2 局限与未来工作

当前为个人研究 MVP，存在明确局限：来源快照规模有限，语义召回尚未引入向量检索；官方案例库存在访问限制，部分判决难获全文；尚无多人协作、权限与审计日志。未来工作遵循「需求驱动、按需加重」：当来源规模与语义召回需求稳定后再评估 `sqlite-vec` / `LanceDB`；当高频流程稳定后再服务化。在此之前，坚持用尽量少的项目代码约束通用 agent 的执行边界。

8 经验与反思

8.1 「边界优先」带来了什么

以人类定义边界、agent 负责执行的方式推进，本项目得到三点收益。其一是**可审计**：所有判断规则都是纯文本、可 diff、可追溯，任何结论都能回溯到某条配置或某级来源，而非黑箱输出。其二是**低迁移成本**：不绑定 LangChain / FastAPI 运行时，规则与子代理在 Claude Code、Codex 间几乎零成本迁移。其三是**快速迭代**：9 个阶段中多次重构（如阶段七的配置分离）都由 agent 在数小时内完成，人类只需把关方向与边界。

8.2 什么是困难的

实践也暴露了这种范式的真实代价。首先，**边界必须写得足够显式**——一旦规则含糊，agent 就会自由发挥、把问题越分越散，因此 `config/` 与 `principles.md` 需要人类反复打磨收紧。其次，**校验不可省略**：检索与生成可以交给 agent，但「结论是否被正确来源支撑」必须由独立的 `citation-verifier` 把关，这也是项目把「找证据」与「下判断」刻意分开的原因。最后，轻量与能力之间需要权衡，因此项目用「需求驱动、按需加重」作为加功能的准绳，避免过早工程化。

8.3 一个递归的观察

有意思的是，3wagent 既用 agent 做政策研究，又由 agent 开发完成——同一套「人定义边界、agent 执行」的纪律在两个层面同时成立。这让本项目本身成为该协作范式的一个可复现案例：人类的价值不在于敲下多少行代码，而在于把领域判断与工程约束准确地表达为机器可执行的边界。

9 总结

3wagent 验证了一种垂直领域的人机协作范式：**人类负责定义架构与能力边界，AI Agent 负责高效执行开发与研究**。项目把跨境政策研究中最值钱的领域判断——分类路由、来源分级、输出契约、时效校验、非目标——显式沉淀为可审计、可复用的配置与规则；把检索、校验、分析、报告生成交给以 Claude Code 为载体的 agent 完成。这种分工既发挥了 agent 的执行效率，又通过人类显式设定的边界守住了可信赖与可审计。最终形态是一个小而稳的研究工作区：用尽量少的代码，把通用 agent 的能力约束在一条清晰、可复用、可留档的跨境政策研究流程里。

附：项目文档见 `README.md` / `README.zh-CN.md`、`docs/architecture.md`、`docs/rag-mcp-design.md` 与 `CHANGELOG.md`；本报告 LaTeX 源码位于 `docs/course-report/`。